



Database Systems Final Assessment - ISYS2014/ISYS5008 2026 Trimester S2/T3

21564668

NATHAN ISAAC (STUDENT)

Introduction / Overview

This project involves the design and implementation of a cricket database system. The system stores information about matches, players, tournaments, stadiums and scores. The goal of this project was to demonstrate design, sql, sql querying database concepts.

Design of the Database

Entities identified:

Tournament: Stores cricket series (World Cup, Ashes, etc.)

Team : Countries or teams
playing

Player: Individual players

Match: Each cricket match

Stadium: Where matches are played

Score: Match results

Justification of Entities, Relationships, and Data Types

These identities were identified as they are some of the most essential parts of a cricket match. It was chosen based on how data is naturally organized in such systems.

Tournament: Stores cricket series (World Cup, Ashes, etc.) This was made to represent a series such as ICC World cup Series etc.

Team: Countries or teams playing. This represents national or professional teams playing in these tournaments. These are essential for any match.

Player: Individual players. Stores their details such as player ID, name, date of birth and role. This allows an organized method of keeping track of players or their teams.

Match: Each cricket match. This is an essential part of the system as this stores information about the matches such as when it takes place, or where it takes place. It also helps in keeping track of scores in a tournament for different teams.

Stadium: Where matches are played. This stores information about where the match takes place. Specifically, the location.

Score: Match results. This is essential as there is a possibility of mixing up scores and causing unfair situations. It is made a weak identity as it depends on both **Match** and **Team** for its identification. A composite key (match_id, team_id) ensures that each team's performance in a match is uniquely recorded.

Justification for relationships:

Team – Player (1:N)

One team has many players, but each player belongs to only one team. This reflects real-world team membership.

Tournament – Match (1:N)

A tournament consists of multiple matches, but each match belongs to one tournament.

Stadium – Match (1:N)

A stadium can host multiple matches, but each match is played at one stadium.

Match – Score (1:N)

Each match generates multiple score records (one per team). This relationship supports recording both teams' performances.

Team – Match (M:N resolved via Score)

Teams participate in many matches, and each match involves multiple teams. This many-to-many relationship is resolved using the Score entity.

Participation Constraints

1. Team – Player Relationship

Relationship: Team has Players

- **Player participation: Total**
- **Team participation: Partial**

2. Tournament – Match Relationship

Relationship: Tournament contains Matches

- **Match participation: Total**
- **Tournament participation: Partial**

3. Match – Stadium Relationship

Relationship: Match played at Stadium

- **Match participation: Total**
- **Stadium participation: Partial**

4. Match – Score Relationship

Relationship: Match has Scores

- **Score participation: Total**
- **Match participation: Partial**

5. Team – Match Relationship

Relationship: Teams participate in Matches

- Team participation: Partial
- Match participation: Total

Entities And Their Attributes

Tournament: tournament_id (PK) tournament_name format (test, ODI, T20) year host_country	Team team_id (PK) team_name country coach
Player player_id (PK) player_name date_of_birth role team_id (FK)	Match match_id (PK) tournament_id (FK) stadium_id (FK) match_date
Stadium stadium_id (PK) stadium_name city country capacity	Score (Weak identity) (match_id, team_id) as PK (composite key) match_id (FK) team_id (FK) runs wickets overs

Data Types Justification:

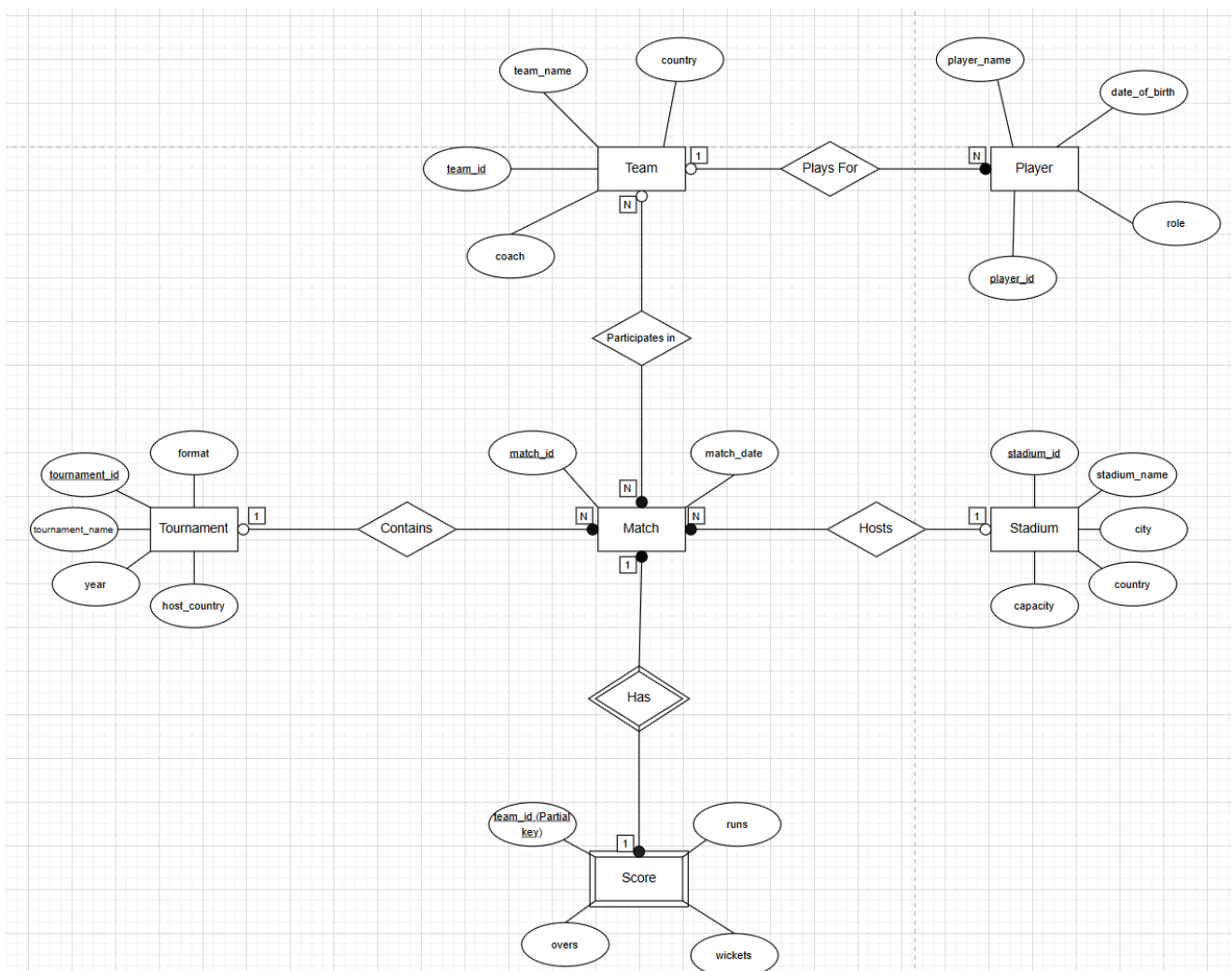
INT was used for IDs (e.g., team_id, match_id) to allow fast indexing and uniqueness.

VARCHAR was used for names and textual attributes such as team_name and stadium_name to allow flexible string storage.

DATE was used for attributes such as match_date and date_of_birth to enable date-based queries and calculations.

DECIMAL was used for overs to accurately store fractional values (e.g., 49.3 overs).

ER DIAGRAM:



(Circles indicate cardinality (total or partial participation) with full black circles representing Total and empty white circles representing Partial participation.)

Relational Schema:

Tournament (tournament_id, tournament_name, format, year, host_country)

Team (team_id, team_name, country, coach)

Player (player_id, player_name, date_of_birth, role, team_id)
FK team_id REF TEAM(team_id)

Stadium (stadium_id, stadium_name, city, country, capacity)

Matches (match_id, tournament_id, stadium_id, match_date)
FK tournament_id REF TOURNAMENT(tournament_id)
FK stadium_id REF STADIUM(stadium_id)

Score (match_id, team_id, runs, wickets, overs)
FK match_id REF MATCHES(match_id)
FK team_id REF TEAM(team_id)

Assumptions made:

- Each player belongs to only one team at a time.
- Each match is played between two teams only.
- Each match is played at a one stadium.
- Scores are recorded per team per match, resulting in two score records per match.
- Tournament data is simple and does not include detailed scheduling or stages.
- Player statistics are limited to basic attributes and do not include performance indicators such as runs scored or wickets taken for each individual player.

Part 1c: Data Description

Tournament Table:

Attribute	Data Type	Description	Constraints
tournament_id	INT	A unique identifier for each tournament	PK, NOT NULL
tournament_name	VARCHAR(50)	Name of the tournament	NOT NULL
format	VARCHAR(10)	Format of the match (Test, ODI, T20)	NOT NULL
year	INT	Year that the tournament takes place	NOT NULL
host_country	VARCHAR(50)	Country that is hosting the tournament	

Team Table:

Attribute	Data Type	Description	Constraints
team_id	INT	A unique identifier for each team	PK, NOT NULL
team_name	VARCHAR(50)	Name of the team	NOT NULL
country	VARCHAR(50)	Country that the team represents	NOT NULL
coach	VARCHAR(50)	Name of the team's coach	

Player table

Attribute	Data Type	Description	Constraints
player_id	INT	A unique ID for each player	PK, NOT NULL
player_name	VARCHAR(50)	Full name of the player	NOT NULL
date_of_birth	DATE	Player's date of birth	
role	VARCHAR(20)	Role / Position (Batsman, Bowler, etc.)	
team_id	INT	Team the player belongs to	FK, NOT NULL

Stadium table:

Attribute	Data Type	Description	Constraints
stadium_id	INT	A unique ID for each stadium	PK, NOT NULL
stadium_name	VARCHAR(50)	Name of the stadium	NOT NULL
city	VARCHAR(50)	Name of the city that the stadium is located in	NOT NULL
country	VARCHAR(20)	Country that the stadium is located in	NOT NULL
capacity	INT	Maximum number of people that the stadium can hold	

Matches table:

Attribute	Data Type	Description	Constraints
-----------	-----------	-------------	-------------

match_id	INT	A unique ID for each match	PK, NOT NULL
tournament_id	INT	Tournament that the match belongs to	FK, NOT NULL
stadium_id	INT	Stadium where the match is held in	FK, NOT NULL
match_date	DATE	Date in which the match takes place	FK, NOT NULL

Score table (Weak identity):

Attribute	Data Type	Description	Constraints
match_id	INT	Identifies match that the scores belong to	PK, FK, NOT NULL
team_id	INT	Team identifier	PK, FK, NOT NULL
Runs	INT	Total runs scored	
wickets	INT	Total number of wickets lost	
overs	DECIMAL(4, 1)	Number of overs played	

Implementation of the Database

Note: When implementing, "CREATE TABLE Isaac_21564668" sql statement would not work so for the project table "dswork" was used as instructed by the lecturer.

Each table was created with appropriate primary keys and foreign key constraints to ensure referential integrity between related entities.

When creating all tables I initially did not have an associative identity called Match_Team. I identified a many-to-many relationship between Match and Team, as each match involves multiple teams and each team participates in multiple matches.

To properly model this relationship, I introduced Match_Team. This entity allows the system to record team participation in matches independently of performance data.

Match_Team -> participation

Score -> performance

Evidence of Implementation

Inserting Data

To insert data about matches I found data that is relevant to the data I need to insert. The two CSV files that provided me the data I used in this project is from:

<https://www.kaggle.com/datasets/mahendran1/icc-cricket>

Data was extracted from CSV files

Some of the data was also compiled from multiple online sources as the csv files seemed to only contain data specifically for ODI Matches.

Online external sources:

- <https://www.espncricinfo.com>
- <https://www.cricbuzz.com>

For the online sources I could not find match IDs, so I logically assigned each match that didn't have one, its own match ID.

Use of the Database

Design and implementation of the queries

Query 1:

This query gets all matches along with a formatted match date and the full stadium location. It uses date format and string concatenation to make it more readable.

This is useful for presenting match schedules in a user-friendly format, making it easier to understand when and where matches are played.

```
-- Query 1: List all matches with formatted date and stadium location
-- Shows use of DATE_FORMAT and string concatenation
SELECT
    m.match_id,
    DATE_FORMAT(m.match_date, '%W, %M %d, %Y') AS formatted_date,
    CONCAT(s.stadium_name, ', ', s.city, ', ', s.country) AS stadium_location
FROM Matches m
JOIN Stadium s ON m.stadium_id = s.stadium_id
ORDER BY m.match_date;
```

Query 2:

This query gets players born after the year 1985 and calculates their age in the present time using TIMESTAMPDIFF.

Helps keep track and analyse younger players, which can be used for scouting.

```
-- Query 2: Find players born after 1985 with their age calculated
-- Shows use of TIMESTAMPDIFF and date functions
SELECT
    player_name,
    date_of_birth,
    TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) AS age,
    UPPER(role) AS role_uppercase
FROM Player
WHERE date_of_birth > '1985-01-01'
ORDER BY age;
```

Query 3:

This query organizes scores using CASE statements. The scores are organized into three different categories, High, Medium or Low.

This helps in keeping track of match performance levels quickly.

```
-- Query 4: Display match scores with conditional formatting
-- Shows CASE statements and numeric formatting
SELECT
    s.match_id,
    s.team_id,
    s.runs,
    s.wickets,
    s.overs,
    CONCAT(s.runs, '/', s.wickets) AS score_summary,
    CASE
        WHEN s.runs >= 300 THEN 'High Score'
        WHEN s.runs BETWEEN 200 AND 299 THEN 'Medium Score'
        ELSE 'Low Score'
    END AS score_category
FROM Score s
WHERE s.overs >= 20.0;
```

Query 4:

Extracts month, date, year and quarter using functions.

Enables time-based analysis of matches. It helps see seasonal trends etc.

```
-- Query 5: Find matches from current year with month extraction
-- Shows YEAR() and MONTH() functions
SELECT
    match_id,
    match_date,
    YEAR(match_date) AS year,
    MONTHNAME(match_date) AS month_name,
    QUARTER(match_date) AS quarter
FROM Matches
WHERE YEAR(match_date) = YEAR(CURDATE())
ORDER BY match_date;
```

Query 5:

This query calculates total runs, average runs, highest score, and number of matches played by teams.

Gives an overview of a team and its performance.

```
-- Query 6: Team performance - total runs, average runs, and match count
-- Shows GROUP BY, aggregate functions, and multiple joins
SELECT
    t.team_name,
    t.country,
    COUNT(DISTINCT s.match_id) AS matches_played,
    SUM(s.runs) AS total_runs,
    AVG(s.runs) AS avg_runs_per_match,
    MAX(s.runs) AS highest_score,
    SUM(CASE WHEN s.wickets = 10 THEN 1 ELSE 0 END) AS all_outs_matches
FROM Team t
LEFT JOIN Score s ON t.team_id = s.team_id
GROUP BY t.team_id, t.team_name, t.country
HAVING matches_played > 0
ORDER BY total_runs DESC;
```

Query 6:

This query finds teams that scored above the average runs across all matches using a subquery.

This query helps in identifying stronger teams, with better scores.

```
-- Query 7: Find matches where both teams scored above average
-- Shows subquery with aggregate, HAVING clause
SELECT
    s.match_id,
    t.team_name,
    s.runs,
    s.wickets,
    (SELECT AVG(runs) FROM Score) AS tournament_avg_score
FROM Score s
JOIN Team t ON s.team_id = t.team_id
WHERE s.runs > (SELECT AVG(runs) FROM Score)
ORDER BY s.match_id, s.runs DESC;
```

Query 7:

This query puts the player with its corresponding team. It also calculates the number of matches that the player has played from all the data given. This shows which players are currently active in teams, making it easy to players and their teams.

```
-- Query 8: Players with their team details and total team matches
-- Shows correlated subquery and multiple joins
SELECT
    p.player_name,
    p.role,
    t.team_name,
    t.country,
    (SELECT COUNT(DISTINCT match_id)
     FROM Score s
     WHERE s.team_id = t.team_id) AS team_matches_played
FROM Player p
JOIN Team t ON p.team_id = t.team_id
WHERE p.player_id IN (
    SELECT player_id FROM Player WHERE role LIKE '%batsman%'
)
ORDER BY t.team_name, p.player_name;
```

Query 8:

This query calculates the number of matches played at each stadium. This helps understand which stadiums host the most matches and where teams tend to score higher runs.

```
-- Query 9: Stadium utilization with match counts and venue statistics
-- Shows LEFT JOIN, GROUP BY with multiple aggregates, subquery
SELECT
    s.stadium_name,
    s.city,
    s.country,
    s.capacity,
    COUNT(m.match_id) AS matches_hosted,
    COALESCE(SUM(sc.runs), 0) AS total_runs_scored,
    AVG(sc.runs) AS avg_score_at_venue,
    (SELECT COUNT(DISTINCT team_id)
     FROM Score sc2
     WHERE sc2.match_id IN (SELECT match_id FROM Matches m2 WHERE m2.stadium_id = s.stadium_id)
    ) AS distinct_teams_played
FROM Stadium s
LEFT JOIN Matches m ON s.stadium_id = m.stadium_id
LEFT JOIN Score sc ON m.match_id = sc.match_id
GROUP BY s.stadium_id, s.stadium_name, s.city, s.country, s.capacity
ORDER BY matches_hosted DESC, avg_score_at_venue DESC;
```

Query 9:

This query gives stats for each tournament, including average runs and total matches played. This is useful for analysing rivalries and direct competition between teams.

```
-- Query 10: Tournament analysis with match statistics
-- Shows multiple joins, nested aggregates, and complex grouping
SELECT
    t.tournament_name,
    t.format,
    t.year,
    COUNT(DISTINCT m.match_id) AS total_matches,
    AVG(score_stats.total_runs_per_match) AS avg_runs_per_match,
    MAX(score_stats.total_runs_per_match) AS highest_match_total,
    MIN(score_stats.total_runs_per_match) AS lowest_match_total
FROM Tournament t
LEFT JOIN Matches m ON t.tournament_id = m.tournament_id
LEFT JOIN (
    SELECT
        match_id,
        SUM(runs) AS total_runs_per_match
    FROM Score
    GROUP BY match_id
) score_stats ON m.match_id = score_stats.match_id
GROUP BY t.tournament_id, t.tournament_name, t.format, t.year
HAVING total_matches > 0
ORDER BY t.year DESC, avg_runs_per_match DESC;
```

Query 10:

This query compares two teams' performances in matches using a self-join and calculates wins and average scores.

```
-- Query 12: Head-to-head team comparison
-- Shows self-join and match analysis
SELECT
    t1.team_name AS team1,
    t2.team_name AS team2,
    COUNT(DISTINCT s1.match_id) AS matches_played,
    AVG(s1.runs) AS team1_avg_score,
    AVG(s2.runs) AS team2_avg_score,
    SUM(CASE WHEN s1.runs > s2.runs THEN 1 ELSE 0 END) AS team1_wins,
    SUM(CASE WHEN s2.runs > s1.runs THEN 1 ELSE 0 END) AS team2_wins
FROM Score s1
JOIN Score s2 ON s1.match_id = s2.match_id AND s1.team_id < s2.team_id
JOIN Team t1 ON s1.team_id = t1.team_id
JOIN Team t2 ON s2.team_id = t2.team_id
GROUP BY t1.team_name, t2.team_name
HAVING matches_played >= 1
ORDER BY matches_played DESC;
```

Design and Implementation of the Advanced Features

When implementing advanced features, I initially intended to include Triggers but ended up not using them due to errors that I ran into related to permission problems:

“ERROR 1419 (HY000): You do not have the SUPER privilege and binary logging is enabled”

1) Stored Procedure:

- a. **GetPlayersByTeam:** Gets all players belonging to a specific team. You have to input a team_ID as a parameter and it will return the player names and roles.
- b. **GetMatchWinner:** Determines the winner after comparing stats such as runs, wickets and overs scored by both teams.

2) Views:

- TeamTotalRuns: This view calculates total runs and matches played for each team across all matches.
- Highscores: This shows all matches where a team scored 300 or more runs.

3) Indexes:

- Idx_player_team: Created on the team_id column in the Player table.
- Idx_score_match: Created on the match_id column in the Score table

Output of all the features:

```
+-----+-----+
| player_name | role |
+-----+-----+
| Pat Cummins | Bowler |
| Steve Smith | Batsman |
| David Warner | Batsman |
| Mitchell Starc | Bowler |
| Allan Border | Batsman |
+-----+-----+
5 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

+-----+-----+
| team_name | runs |
+-----+-----+
| India | 201 |
+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

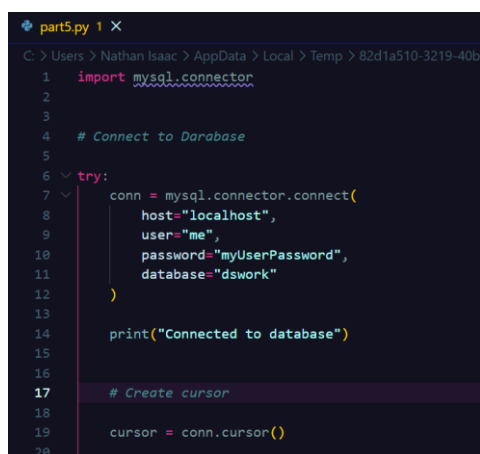
+-----+-----+-----+
| team_name | total_runs | matches_played |
+-----+-----+-----+
| Australia | 3513 | 16 |
| India | 3783 | 20 |
| England | 1289 | 5 |
| South Africa | 1326 | 7 |
| New Zealand | 2716 | 16 |
| Pakistan | 2080 | 11 |
| Sri Lanka | 1933 | 10 |
| West Indies | 175 | 1 |
+-----+-----+-----+
8 rows in set (0.00 sec)

+-----+-----+-----+-----+
| match_id | team_name | runs | wickets |
+-----+-----+-----+-----+
| 7 | Australia | 469 | 10 |
| 5 | Australia | 386 | 10 |
| 8 | India | 356 | 2 |
| 6 | England | 325 | 10 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

Database Connectivity and Python Implementation

Connection Setup:

A screenshot of a code editor window titled 'part5.py 1 X'. The editor shows a Python script for connecting to a MySQL database. The script includes an import statement for 'mysql.connector', a comment '# Connect to Database', a try block containing a 'connect()' call with parameters for host, user, password, and database, a print statement, and a cursor creation section. The file path in the background is 'C:\> Users > Nathan Isaac > AppData > Local > Temp > 82d1a510-3219-40b...'.

```
1 import mysql.connector
2
3
4 # Connect to Database
5
6 try:
7     conn = mysql.connector.connect(
8         host="localhost",
9         user="me",
10        password="myUserPassword",
11        database="dswork"
12    )
13
14    print("Connected to database")
15
16
17 # Create cursor
18
19 cursor = conn.cursor()
20
```

Select Query:

```
# SELECT query

print("\n--- Teams in Database ---")
select_query = "SELECT * FROM Team"
cursor.execute(select_query)

rows = cursor.fetchall() # Step 5: fetch results
for row in rows:
    print(row)
```

This query displays all teams stored in a database

```
STUDENT\21564668@v-2204-H05-012:~/Isaac_21564668/Isaac_21564668
Connected to database

--- Teams in Database ---
(1, 'Australia', 'Australia', 'Andrew McDonald')
(2, 'India', 'India', 'Rahul Dravid')
(3, 'England', 'England', 'Brendon McCullum')
(4, 'South Africa', 'South Africa', 'Rob Walter')
(5, 'New Zealand', 'New Zealand', 'Gary Stead')
(6, 'Pakistan', 'Pakistan', 'Grant Bradburn')
(7, 'Sri Lanka', 'Sri Lanka', 'Chris Silverwood')
(8, 'West Indies', 'West Indies', 'Daren Sammy')
(9, 'Afghanistan', 'Afghanistan', 'Jonathan Trott')
(10, 'Bangladesh', 'Bangladesh', 'Chandika Hathurusingha')
```

Insert Operation:

This insert operation adds a new team

```
# INSERT
print("\n--- Inserting new team ---")

insert_query = "INSERT INTO Team (team_id, team_name, country, coach) VALUES (%s, %s, %s, %s)"
data = (11, "New Zealand", "New Zealand", "Gary Stead")

cursor.execute(insert_query, data)
conn.commit()

print("Inserted successfully")
```

Output:

```
--- Inserting new team ---
Inserted successfully
```

Parameterized SELECT Query – Players by Team:

To demonstrate secure query execution using parameters:

```
# SELECT with parameter

print("\n--- Players from Team 1 ---")

select_param_query = "SELECT player_name FROM Player WHERE team_id = %s"
cursor.execute(select_param_query, (1,))

for row in cursor:
    print(row)
```

Output:

```
--- Players from Team 1 ---
('Pat Cummins',)
('Steve Smith',)
('David Warner',)
('Mitchell Starc',)
('Allan Border',)
```

Connection closed

```
STUDENT\21564668@v-2204-H05-012:~/Isaac_21564668/Isaac_21564668$
```

Closing the connection:

Output:

```
#Close connection

cursor.close()
conn.close()
print("\nConnection closed")
```

Error handling:

Output:

```
except mysql.connector.Error as err:
    print("Error:", err)
```

Discussion

In this assignment I successfully designed and implemented a relational database for a cricket system. The database includes 8 tables: Tournament, Matches, Team, Player, Stadium and Score. Data was added to these tables through CSV files and other online sources.

I made 11 queries from SELECT statements to more complex ones using joins etc. The advanced features I used consist of stored procedures, indexes, and views. I attempted to implement triggers but came across some errors that I could not get through. Finally, I implemented python connectivity to a database to do read and write operations.

Challenges Faces:

- Online sources didn't provide data related to some of the attributes I included such as match_id, which was included in the CSV files but not the other online sources. Therefore, I logically assigned my own match_ids.
- As mentioned before, I was not able to implement triggers due to errors related to permissions.

Limitations:

The database does not in any way track players individual info such as runs, overs and wickets. Instead, it consists of total runs, total wickets or total overs.

Furthermore, there are more attributes or entities that might have been useful that I could have included but did not as it was too far into the project to suddenly implement them.